Повышение квалификации специалистов по информационной безопасности

МЕТОДИЧЕСКИЕ УКАЗАНИЯ
К ПРАКТИЧЕСКОМУ ЗАНЯТИЮ 3

по курсу
«Методы и принципы проектирования современных защищённых систем. Применение механизмов искусственного интеллекта для анализа данных при построении систем информационной безопасности»

ОБНАРУЖЕНИЕ АТАК ПО ЖУРНАЛУ СОБЫТИЙ

Создание стенда, разработка правил, обнаружение

(2 академических часа)

2026

СОДЕРЖАНИЕ

Цель и порядок работы 3
Шаг 1. Создание виртуальной машины 4
Шаг 2. Какие журналы ведёт система 5
Шаг 3. Docker и уязвимое приложение 6
Шаг 4. Последовательная атака в браузере 7
4.1. Подбор пароля 7
4.2. Распыление пароля 7
4.3. Поиск чужих заказов 8
4.4. Отключение MFA 8
Шаг 5. Журнал и первое правило обнаружения 9
Шаг 6. Атака и её обнаружение 10
Базовые правила 10
Подбор пароля 10
Распыление паролей 10
Успешный вход после серии неудач 11
Подготовьте контрольный журнал 11
Примените все правила 12
Шаг 7. Настройка и отчёт 13
Дополнительное задание: поймать IDOR 14
Если что-то не получилось 15

Цель и порядок работы

Занятие проводит слушателя через весь путь от чистой системы до обнаружения атаки: создание виртуальной машины с Ubuntu, изучение её журналов, развёртывание уязвимого веб-приложения в Docker, ручное исследование приложения в браузере, разработка правил обнаружения в формате Sigma, проведение атаки и её обнаружение, настройка правила для снижения числа ложных срабатываний.

Результат занятия — набор правил обнаружения, результаты их срабатываний и предложение по снижению ложных срабатываний. Они оформляются в отчёт по шаблону `report-template.md`.

Каждый шаг завершается **ожидаемым результатом**. Прежде чем переходить к следующему шагу, сверьтесь с ним.

Для работы понадобятся гипервизор (VirtualBox или аналог), установочный образ Ubuntu Server 24.04 и каталог материалов практики — их выдаёт преподаватель. Из программ на виртуальной машине используются Docker (ставится на шаге 3) и Python 3 (входит в состав Ubuntu). Команды, набираемые в терминале, и содержимое файлов правил приведены в тексте полностью.

Шаг 1. Создание виртуальной машины

Установку можно начать и, пока она идёт, читать следующий шаг.

1. В гипервизоре создайте виртуальную машину: тип — Linux, версия — Ubuntu (64-bit), память — 2048 МБ, диск — 20 ГБ.
2. Подключите установочный образ Ubuntu Server к виртуальному приводу и запустите машину.
3. Пройдите установщик: раскладка по умолчанию, установка на весь диск, имя пользователя и пароль — на ваш выбор (запомните их). На экране «SSH Setup» включите **Install OpenSSH server**.
4. Дождитесь окончания установки, перезагрузите машину и войдите под своей учётной записью.

Проверьте версию системы:

```
lsb_release -d
```

Ожидаемый результат: строка вида `Description: Ubuntu 24.04 LTS`. Вы вошли в систему, виден приглашение командной строки.

Шаг 2. Какие журналы ведёт система

Ubuntu регистрирует события безопасности ещё до установки каких-либо приложений. Посмотрим их.

Последние события входа в систему:

```
journalctl -n 20
sudo cat /var/log/auth.log | tail -n 20
```

Создайте событие безопасности вручную — неудачный вход по `sudo` с неверным паролем:

```
sudo -k            # забыть сохранённый пароль
sudo whoami        # введите НЕВЕРНЫЙ пароль один раз, затем верный
```

Найдите эту неудачу в журнале:

```
sudo grep "authentication failure" /var/log/auth.log | tail -n 3
```

Ожидаемый результат: в выводе есть строка с `authentication failure` и вашим именем пользователя. Это событие безопасности, записанное операционной системой.

Вопрос в отчёт (п. 1): какое поле строки показывает, с какого терминала или адреса пришёл неудачный вход?

Шаг 3. Docker и уязвимое приложение

Установите Docker официальным сценарием:

```
curl -fsSL https://get.docker.com | sudo sh
sudo usermod -aG docker $USER && newgrp docker
docker --version
```

Если в аудитории нет доступа в интернет, преподаватель выдаст готовый пакет или образ; следуйте его указаниям.

Запустите учебное приложение «Магазин» из каталога материалов практики:

```
cd app
docker compose up -d --build
docker compose ps
```

Откройте в браузере `http://localhost:8080`. Вы увидите интерфейс **TraceShop LAB**: слева находится рабочая консоль исследователя, справа — живая телеметрия, сверху — четыре миссии. Если требуется только техническая проверка, откройте `http://localhost:8080/health`.

Нажмите кнопку **Сбросить стенд**, чтобы очистить события от предыдущих запусков.

Ожидаемый результат: `docker compose ps` показывает контейнер `shop-app` в состоянии `Up`, а в браузере открывается TraceShop LAB со счётчиком `0 событий`.

Шаг 4. Последовательная атака в браузере

Приложение записывает каждое значимое действие в `app/logs/events.jsonl`. Миссии на сайте выполняются строго по порядку: следующая рабочая консоль разблокируется только после предыдущей. Счётчики на карточках показывают, чего ещё не хватает.

4.1. Подбор пароля

1. Оставьте учётную запись `anna` и адрес `203.0.113.77`.
2. Отправьте пять разных неверных паролей. Можно последовательно использовать кнопки `123456`, `qwerty`, `anna2026`, `sunflower`, `letmein`.
3. По перехваченной записке догадайтесь, что верный пароль — `sunflower7`, и выполните шестую попытку.

Важно: успешный вход, сделанный **до** пяти неудач, не завершает миссию. В этом случае сайт объяснит причину; накопите пять отказов и повторите успешный вход.

Сопоставьте живую телеметрию с исходным журналом:

```
grep '"event_type": "auth.login"' logs/events.jsonl | tail -n 2
```

Ожидаемый результат: карточка «Получить доступ как anna» зелёная, а блок распыления разблокирован. В журнале видны `auth.login / failure` и следующий за серией `auth.login / success`.

Вопрос в отчёт (п. 2): выпишите значения `event_type`, `outcome`, `actor.id` из записи о неудачном входе.

4.2. Распыление пароля

Оставьте пароль `123456` и адрес `198.51.100.23`. Отправляйте попытки для предлагаемых сайтом логинов; после каждой попытки поле автоматически переключается на следующий. Нужны восемь разных учётных записей.

Ожидаемый результат: прогресс достигает `8/8`, карточка распыления становится зелёной, разблокируется перебор заказов.

4.3. Поиск чужих заказов

Действуйте как anna. Начните с заказа 1001 и нажимайте **Открыть заказ**: после каждого запроса номер увеличивается автоматически. Заказы 1001 и 1002 принадлежат anna; начиная с 1003 появляются отказы. Накопите пять отказов.

Ожидаемый результат: после проверки заказов 1001–1007 карточка становится зелёной и разблокируется отключение MFA.

4.4. Отключение MFA

Оставьте пользователя anna, а поле заявки — пустым. Нажмите **Отключить MFA**.

Ожидаемый результат: все четыре карточки зелёные. В телеметрии 22 события: 13 ошибок входа, 5 отказов доступа и одно отключение MFA; остальные события — успешный вход и чтение двух своих заказов.

Шаг 5. Журнал и первое правило обнаружения

Правило обнаружения — условие над записями журнала. Запишем его в формате Sigma. Начнём с простого правила по одиночному событию: кто-то отключил второй фактор аутентификации.

Команды ниже выполняются из каталога материалов практики. Перейдите в каталог проверки правил и создайте каталог для правил:

```
cd ../detect
mkdir -p rules
```

Создайте файл `rules/01_mfa.yml`:

```
title: Отключение второго фактора без заявки
id: 00000000-0000-0000-0000-0000000000b1
name: mfa_disabled
logsource: {product: shop-app, service: auth}
detection:
    selection:
        event_type: auth.mfa.disable
        outcome: success
    filter_ticket:
        context.ticket_id|exists: true
    condition: selection and not filter_ticket
level: high
tags: [attack.credential-access, attack.t1556.006]
```

Правило срабатывает на событие `auth.mfa.disable`, кроме случаев, когда отключение оформлено заявкой (поле `context.ticket_id`).

Событие отключения MFA уже создано последней миссией шага 4. Примените к журналу новое правило:

```
python3 detect.py --events ../app/logs/events.jsonl --rules rules
```

Ожидаемый результат: программа выводит `сигналов: 1` и строку с названием правила «Отключение второго фактора без заявки».

Шаг 6. Атака и её обнаружение

Напишем правила для трёх атак и проверим их на настоящей активности.

Базовые правила

Корреляционные правила ссылаются на простые. Создайте файл `rules/00_base.yml`:

```yaml
title: Неудачный вход
id: 00000000-0000-0000-0000-0000000000a1
name: failed_login
logsource: {product: shop-app, service: auth}
detection:
    selection: {event_type: auth.login, outcome: failure}
    condition: selection
---
title: Успешный вход
id: 00000000-0000-0000-0000-0000000000a2
name: successful_login
logsource: {product: shop-app, service: auth}
detection:
    selection: {event_type: auth.login, outcome: success}
    condition: selection
```

Подбор пароля

Серия неудачных входов в одну учётную запись. Создайте `rules/02_bruteforce.yml`:

```yaml
title: Подбор пароля к учётной записи
id: 00000000-0000-0000-0000-0000000000b2
name: bruteforce_account
correlation:
    type: event_count          # считаем события
    rules: [failed_login]      # неудачные входы
    group-by: [actor.id]       # отдельно по каждой учётной записи
    timespan: 10m              # за 10 минут
    condition: {gte: 5}        # 5 и более
level: medium
tags: [attack.credential-access, attack.t1110.001]
```

Распыление паролей

Один адрес пробует один пароль против многих учётных записей. Создайте `rules/03_spraying.yml`:

```yaml
title: Распыление паролей с одного адреса
id: 00000000-0000-0000-0000-0000000000b3
name: password_spraying
correlation:
    type: value_count          # считаем РАЗНЫЕ значения поля
    rules: [failed_login]
    group-by: [source_ip]      # по адресу источника
    timespan: 30m
    condition: {field: actor.id, gte: 8}   # 8 и более разных учётных записей
level: high
tags: [attack.credential-access, attack.t1110.003]
```

Успешный вход после серии неудач

Признак успешного подбора. Создайте `rules/04_sequence.yml`:

```yaml
title: Успешный вход после серии неудачных попыток
id: 00000000-0000-0000-0000-0000000000b4
correlation:
    type: temporal_ordered     # события в заданном порядке
    rules: [failed_login_burst, successful_login]
    group-by: [actor.id]
    timespan: 15m
level: high
tags: [attack.initial-access, attack.t1078]
---
title: Три и более неудачных входа
id: 00000000-0000-0000-0000-0000000000b5
name: failed_login_burst
correlation:
    type: event_count
    rules: [failed_login]
    group-by: [actor.id]
    timespan: 10m
    condition: {gte: 3}
```

Подготовьте контрольный журнал

Ручной квест показал механику атак, но его журнал зависит от числа ваших попыток. Для воспроизводимой проверки нажмите в браузере **Сбросить стенд**, затем запустите `simulate.py`. Скрипт повторяет те же четыре атаки и добавляет обычную работу пользователей:

```
cd ../attack
python3 simulate.py --url http://localhost:8080
```

Примените все правила

```
cd ../detect
python3 detect.py --events ../app/logs/events.jsonl --rules rules --out
alerts.jsonl
```

Ожидаемый результат: программа сообщает `Событий: 39` и `сигналов: 5`. Срабатывают правила подбора пароля (учётная запись `anna`), распыления (один адрес), отключения второго фактора и **два** срабатывания правила «успешный вход после серии неудач» — для `anna` и для `boris`. В браузере эти же 39 событий остаются доступными для визуального разбора.

Вопрос в отчёт (п. 3): какие правила сработали и для каких учётных записей или адресов? Для каждого сигнала укажите, истинный он или ложный.

Шаг 7. Настройка и отчёт

Одно из срабатываний правила «успешный вход после серии неудач» — ложное: пользователь `boris` трижды ошибся в пароле и вошёл с четвёртой попытки (забыл пароль). Это не атака.

Настройте правило: в файле `rules/04_sequence.yml` поднимите порог серии неудач с 3 до 5 (`condition: {gte: 5}` во втором документе) и примените правила заново:

```
python3 detect.py --events ../app/logs/events.jsonl --rules rules
```

Ожидаемый результат: ложное срабатывание для `boris` исчезло; подбор пароля к `anna` (7 неудач) по-прежнему обнаруживается, итого 4 сигнала.

Вопрос в отчёт (п. 4): какова цена этого изменения — какую атаку правило теперь способно пропустить?

Заполните отчёт по шаблону `report-template.md` и сдайте его преподавателю вместе с каталогом `rules/`.

Дополнительное задание: поймать IDOR

Нарушитель перебирал чужие заказы: под учётной записью `anna` обращался к заказам с номерами 1001–1009, получая отказы в доступе (`authz.access` с результатом `failure`). Найдите эту серию сначала в живой телеметрии браузера, затем напишите правило, которое обнаруживает перебор чужих объектов: `event_count` события `authz.access` (неудача) с группировкой по `actor.id`. Сколько отказов в доступе зафиксировано для `anna`?

Если что-то не получилось

Признак	Что сделать
`docker compose ps` не показывает `Up`	Посмотреть журнал контейнера: `docker compose logs`
`сигналов: 0` после атаки	Проверить, что создан `rules/00_base.yml`: без базовых правил корреляции не работают
Непонятная ошибка правила	Проверить отступы в YAML (ровно 4 пробела на уровень) и запустить `python3 detect.py` ещё раз: программа укажет файл с ошибкой